

Creativoid:

Content Creation App with Local Data Management

Jonathan O. Peraza • Scott Feister, Ph.D. • COMP - 499

Introduction

I was inspired to work on this project to help small business owners who might be struggling to reach out to new customers. I wanted to give them the possibility to create something, even with a lack of creativity or know how for creating marketing material for social media interactions. The result was Creativoid, an application that will create the contents of a post to be shared with others.



Figure 1: Home Screen Logo

With this application a business owner can first insert their products and attach any wanted or associated images, tags, descriptions and backgrounds. The app then randomizes all the given information and creates an image along with a caption which can then be posted to any used social media applications.

Methodology

- I started the project by researching and building a local SQLite database using Room persistence libraries with Android Studio.
- I followed up with a small advertising media campaign, taking pictures of the products offered by the small business owner I partnered with.
- Work then started on the development and implementation of the core features for automating content creation and organizing it.
- Feedback on the created content and general use of the application was then gathered from the small business owner.
- Using the feedback given, I further developed the application and polished both the content created, and the user interfaces of the application.
- Finally I added features to allow the application to share the created content with social media applications installed on the device.

Result

- Given the need for the user to input the information of their products, a database management system had to be created.
- Once the data is populated the user has the ability, to choose between three different modes of operations, select from a product, select from a tag or select randomly from all products.
- When the user presses create, the application will randomize all linked data, based on the chosen mode, and generate a preview image. If the preview is to the users liking they can press share creation, to show the preview of the file to be exported.
- Once the export is shared it will open the share menu on their phone and automatically detect any available social media application which can be used.

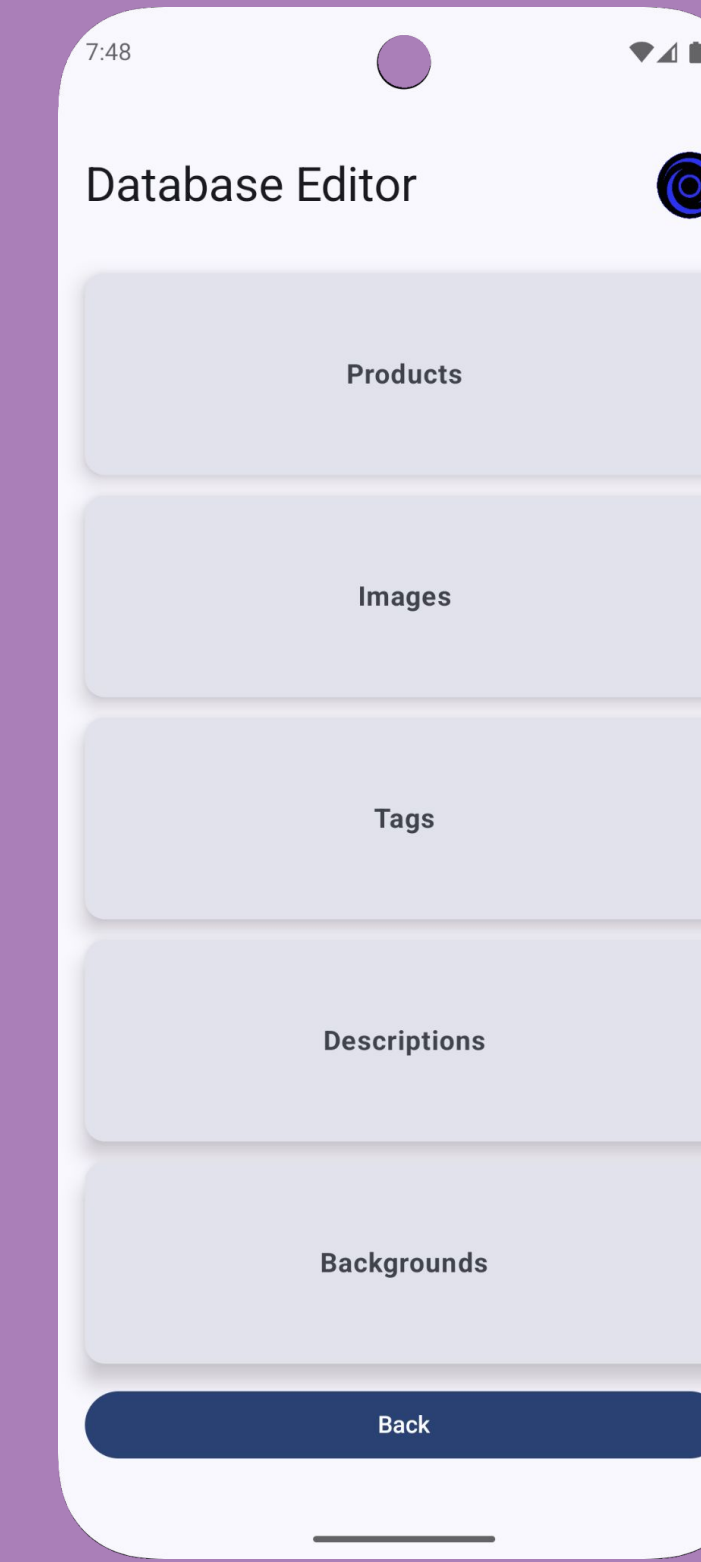


Figure 2: Database Menu

Software

Software used to create the application:

- Android Studio
- Gimp



Social media applications which were tested and compatible:

- Discord
- Facebook
- Instagram
- Threads
- X



Implemented Code

```
//export
fun exportToBitmap(
    context: Context,
    product: Product?,
    image: ProductImage?,
    background: Background?,
    tags: List<Tag>,
    description: Description?
): Bitmap {
    val width = 1080
    val height = 1080

    val bitmap = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888)
    val canvas = android.graphics.Canvas(bitmap)

    background?.let {
        val bg = BitmapFactory.decodeFile(it.backgroundPath)
        bg?.let { bmp ->
            val scale = calculateScaleToCover(
                bmp.width,
                bmp.height,
                width,
                height
            )
            val scaledWidth = bmp.width * scale
            val scaledHeight = bmp.height * scale
            val left = (width - scaledWidth) / 2f
            val top = (height - scaledHeight) / 2f
            val dist = Rect(
                left,
                top,
                right = left + scaledWidth,
                bottom = top + scaledHeight
            )
            canvas.drawBitmap(bmp, dist, dist, null)
        }
    }
    canvas.drawBitmap(bitmap, 0, 0, null, null)
}
```

Figure 3: Code used to export the selected images into a single .png file. This allows the app to save or share the exported image.

Figure 4: Code used to structure the database, allowing the application to locally save any information given to each entity.

```
@Database(entities = [
    Product::class,
    ProductImage::class,
    Tag::class,
    Description::class,
    Background::class
], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
    abstract fun productDao(): ProductDao
    abstract fun imageDao(): ImageDao
    abstract fun tagDao(): TagDao
    abstract fun descriptionDao(): DescriptionDao
    abstract fun backgroundDao(): BackgroundDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "product_database"
                ).build()
                INSTANCE = instance
            }
        }
    }
}
```

Acknowledgements

I would like to thank all of the following:

- Dr. Feister for steering me into the right direction with my project.
- My family and friends for being supportive during my academic journey.
- Fellow classmates for keeping our spirits up during the good and the tough times.

Future Improvements

Future improvements/features on the application could include:

- Automatic or scheduled posting of created content.
- Ability to edit the created images before being exported and shared.
- Integration of the phones camera into the image editor section of the application.
- Listing the application on an app store.